

Higher National School of Artificial Intelligence
(ENSIA)

Big Data Technologies Laboratory
LAB 2: Setting up Hadoop Cluster with Multiple Nodes

Big Data Analysis Report

Students:

BOUHOUIA Yousra – Group G1

Supervisor: F. DAHAK

Academic Year: 2024/2025

Semester: Spring 2025

Algiers, Algeria
February 24, 2026

C. PHASE 1: CLUSTER DISCOVERY (20 MINUTES)

Challenge 1.1: Cluster Topology Reconnaissance

Task: Discover the cluster architecture using available commands.

1. NameNode Address and Port

Command:

```
hdfs getconf -confKey fs.defaultFS
```

Output:

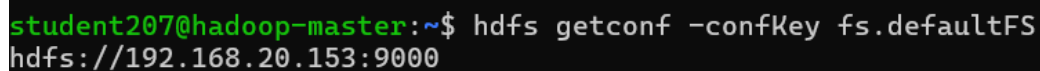
```
hdfs://192.168.20.153:9000
```

Answer:

The HDFS filesystem is managed by the NameNode located at:

- **Address:** 192.168.20.153
- **Port:** 9000

Screenshot:



```
student207@hadoop-master:~$ hdfs getconf -confKey fs.defaultFS
hdfs://192.168.20.153:9000
```

Figure 1: Execution of fs.defaultFS command

2. Live DataNodes

Command:

```
hdfs dfsadmin -report
```

Based on the report output, the cluster capacity and statistics were displayed.

Answer:

- **Number of active DataNodes:** 0

Screenshot:

```
student207@hadoop-master:~$ hdfs dfsadmin -report
Configured Capacity: 2075734720512 (1.89 TB)
Present Capacity: 2047010852973 (1.86 TB)
DFS Remaining: 2020740554752 (1.84 TB)
DFS Used: 26270298221 (24.47 GB)
DFS Used%: 1.28%
Replicated Blocks:
    Under replicated blocks: 312
    Blocks with corrupt replicas: 0
    Missing blocks: 0
    Missing blocks (with replication factor 1): 0
    Low redundancy blocks with highest priority to recover: 0
    Pending deletion blocks: 0
Erasure Coded Block Groups:
    Low redundancy block groups: 0
    Block groups with corrupt internal blocks: 0
    Missing block groups: 0
    Low redundancy blocks with highest priority to recover: 0
    Pending deletion blocks: 0
```

Figure 2: HDFS cluster report showing storage capacity, usage statistics, and block health status.

3. HDFS Version and Cluster ID

Command:

`hdfs version`

Output shows:

- Hadoop Version: 3.3.6

Cluster ID can be obtained from the VERSION file or dfsadmin report.

Answer:

- HDFS Version: 3.3.6
- Cluster ID: _____

Screenshot:

```
student207@hadoop-master:~$ hdfs version
Hadoop 3.3.6
Source code repository https://github.com/apache/hadoop.git -r 1be78238728da9266a4f88195058f08fd012bf9c
Compiled by ubuntu on 2023-06-18T08:22Z
Compiled on platform linux-x86_64
Compiled with protoc 3.7.1
From source with checksum 5652179ad55f76cb287d9c633bb53bbd
This command was run using /opt/hadoop-cluster/hadoop/share/hadoop/common/hadoop-common-3.3.6.jar
```

Figure 3: Enter Caption

4. Total Cluster Capacity

Command:

```
hdfs dfsadmin -report
```

Relevant output:

Configured Capacity: 1.89 TB

DFS Used: 24.47 GB

DFS Remaining: 1.84 TB

DFS Used%: 1.28%

Answer:

- **Total Configured Capacity:** 1.89 TB
- **Used Capacity:** 24.47 GB
- **Remaining Capacity:** 1.84 TB
- **Usage Percentage:** 1.28%

Challenge 1.2: Health Diagnostics

5. Missing blocks or corrupt replicas

Command Used:

```
hdfs fsck /user/student207
```

```

student207@hadoop-master:~$ hdfs fsck /user/student207
Connecting to namenode via http://hadoop-master:9870/fsck?ugi=student207&path=%2Fuser%2Fstudent207
FSCK started by student207 (auth:SIMPLE) from /192.168.20.153 for path /user/student207 at Tue Feb 24 13:50:04 UTC 2026

Status: HEALTHY
Number of data-nodes: 4
Number of racks: 1
Total dirs: 1
Total symlinks: 0

Replicated Blocks:
Total size: 0 B
Total files: 0
Total blocks (validated): 0
Minimally replicated blocks: 0
Over-replicated blocks: 0
Under-replicated blocks: 0
Mis-replicated blocks: 0
Default replication factor: 3
Average block replication: 0.0
Missing blocks: 0
Corrupt blocks: 0
Missing replicas: 0
Blocks queued for replication: 0

Erasure Coded Block Groups:
Total size: 0 B
Total files: 0
Total block groups (validated): 0
Minimally erasure-coded block groups: 0
Over-erasure-coded block groups: 0
Under-erasure-coded block groups: 0
Unsatisfactory placement block groups: 0
Average block group size: 0.0
Missing block groups: 0
Corrupt block groups: 0
Missing internal blocks: 0
Blocks queued for replication: 0
FSCK ended at Tue Feb 24 13:50:04 UTC 2026 in 1 milliseconds

```

Figure 4: Filesystem check (fsck) output for /user/student207

Answer:

No missing blocks or corrupt replicas were detected.

Explanation:

The fsck output returned **Status: HEALTHY** with zero corrupt or under-replicated blocks, indicating the filesystem is operating correctly.

6. Safemode Status

Command Used:

```
hdfs dfsadmin -safemode get
```

Answer: Safe mode : OFF

Explanation:

Safe mode is OFF, the cluster is fully operational and allows read/write operations.

```

Last login: Tue Feb 24 13:12:32 2026 from 10.0.0.21:80
student207@hadoop-master:~$ hdfs dfsadmin -safemode get
Safe mode is OFF

```

Figure 5: Enter Caption

7. NameNode Logs Analysis

Command Used:

```
tail -50 $HADOOP_HOME/logs/hadoop-*-namenode-*.log
```

Answer:

Recent NameNode log entries show FSCK activity, permission denials, and replica placement warnings.

Explanation:

The logs record multiple FSCK requests from users and access control exceptions (e.g., WRITE denied on `/user`). They also report a block placement warning (“Not enough replicas”), indicating replication constraints during placement.

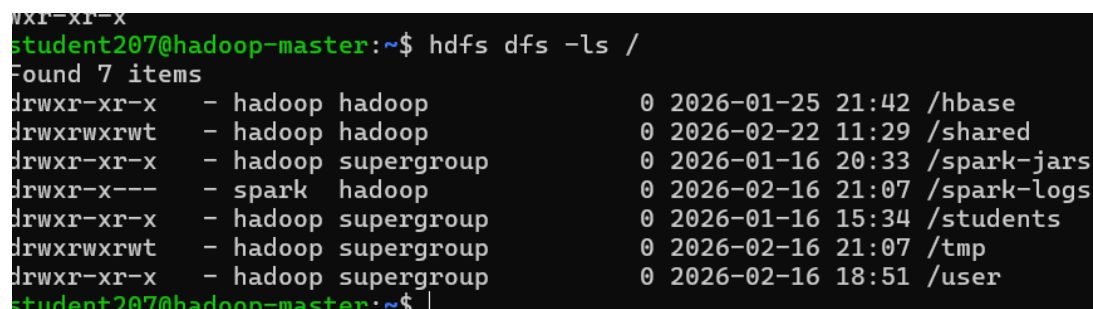
D. PHASE 2: HDFS FILE OPERATIONS (25 MINUTES)

Challenge 2.1: Namespace Navigation

8. Root Directory Contents

Command Used:

```
hdfs dfs -ls /
```



```
student207@hadoop-master:~$ hdfs dfs -ls /
Found 7 items
drwxr-xr-x   - hadoop hadoop          0 2026-01-25 21:42 /hbase
drwxrwxrwt   - hadoop hadoop          0 2026-02-22 11:29 /shared
drwxr-xr-x   - hadoop supergroup      0 2026-01-16 20:33 /spark-jars
drwxr-xr-x   - spark hadoop           0 2026-02-16 21:07 /spark-logs
drwxr-xr-x   - hadoop supergroup      0 2026-01-16 15:34 /students
drwxrwxrwt   - hadoop supergroup      0 2026-02-16 21:07 /tmp
drwxr-xr-x   - hadoop supergroup      0 2026-02-16 18:51 /user
student207@hadoop-master:~$
```

Figure 6: Root directory contents

Answer:

The root directory contains default system folders such as `/user`, `/tmp`, and other service-related directories.

Explanation:

These directories are automatically created during Hadoop setup. They organize user data, temporary files, and system-level services.

9. Personal Workspace

Command Used:

```
hdfs dfs -ls /user/student207
```

```

drwx----- - student290 hadoop_students 0 2026-02-16 15:22 /user/student290
drwx----- - student291 hadoop_students 0 2026-02-16 15:22 /user/student291
drwx----- - student292 hadoop_students 0 2026-02-16 15:22 /user/student292
drwx----- - student293 hadoop_students 0 2026-02-16 15:22 /user/student293
drwx----- - student294 hadoop_students 0 2026-02-16 15:22 /user/student294
drwx----- - student295 hadoop_students 0 2026-02-16 15:23 /user/student295
drwx----- - student296 hadoop_students 0 2026-02-16 15:23 /user/student296
drwx----- - student297 hadoop_students 0 2026-02-16 15:23 /user/student297
drwx----- - student298 hadoop_students 0 2026-02-16 15:23 /user/student298
drwx----- - student299 hadoop_students 0 2026-02-16 15:23 /user/student299
drwx----- - student300 hadoop_students 0 2026-02-24 11:04 /user/student300
drwx----- - teacher1 hadoop_students 0 2026-02-16 18:51 /user/teacher1
drwx----- - teacher2 hadoop_students 0 2026-02-16 18:51 /user/teacher2
student207@hadoop-master:~$ hdfs dfs -ls /user/student207

```

Figure 7: Accessing personal workspace

Answer:

The personal workspace directory `/user/student207` is accessible and used to store user-specific HDFS files.

Explanation:

Each Hadoop user has a dedicated home directory under `/user`. This ensures logical separation and access control of user data.

10. `/tmp` and `/user/hive/warehouse` Permissions

Command Used:

```

hdfs dfs -ls -d /tmp
hdfs dfs -ls -d /user/hive/warehouse

```

```

student207@hadoop-master:~$ hdfs dfs -ls -d /tmp
drwxrwxrwt - hadoop supergroup 0 2026-02-16 21:07 /tmp
student207@hadoop-master:~$ hdfs dfs -ls -d /user/hive/warehouse
drwxrwxrwx - hadoop hadoop 0 2026-02-22 22:46 /user/hive/warehouse

```

Figure 8: Permission verification for `/tmp` and Hive warehouse

Answer:

The `/tmp` directory has permissions `drwxrwxrwt`, while `/user/hive/warehouse` has permissions `drwxrwxrwx`.

Explanation:

The sticky bit (`t`) on `/tmp` allows users to create files but prevents deletion of others' files. The Hive warehouse directory allows full access for Hive operations and table storage.

Challenge 2.2: File Manipulation

```
student207@hadoop-master:~$ hdfs dfs -ls /user/student207/
student207@hadoop-master:~$ hdfs dfs -ls -d /tmp
drwxrwxrwt - hadoop supergroup          0 2026-02-16 21:07 /tmp
student207@hadoop-master:~$ hdfs dfs -ls -d /user/hive/warehouse
drwxrwxrwx - hadoop hadoop              0 2026-02-22 22:46 /user/hive/warehouse
student207@hadoop-master:~$ echo "Hello Hadoop" > file1.txt
echo "Big Data Lab" > file2.txt
dd if=/dev/zero of=largefile.txt bs=1M count=10
10+0 records in
10+0 records out
10485760 bytes (10 MB, 10 MiB) copied, 0.0193466 s, 542 MB/s
student207@hadoop-master:~$ hdfs dfs -put file1.txt /user/student207/
hdfs dfs -put file2.txt /user/student207/
hdfs dfs -put largefile.txt /user/student207/
student207@hadoop-master:~$ hdfs dfs -ls /user/student207/
Found 3 items
-rw-r--r--  3 student207 hadoop_students      13 2026-02-24 14:13 /user/student207/file1.txt
-rw-r--r--  3 student207 hadoop_students      13 2026-02-24 14:13 /user/student207/file2.txt
-rw-r--r--  3 student207 hadoop_students 10485760 2026-02-24 14:13 /user/student207/largefile.txt
student207@hadoop-master:~$ |
```

Figure 9: Enter Caption

11. Upload Test Files

Command Used:

```
hdfs dfs -put file1.txt /user/student207/
hdfs dfs -put file2.txt /user/student207/
hdfs dfs -put largefile.txt /user/student207/
hdfs dfs -ls /user/student207/
```

Answer:

The three files were successfully uploaded to the HDFS home directory and verified using the listing command.

Explanation:

The `-put` command copies local files into HDFS. The `-ls` command confirms their presence and size in HDFS.

12. Replication Factor

Command Used:

```
hdfs dfs -stat %r /user/student207/file1.txt
```

Answer:

The default replication factor is 3.

Explanation:

HDFS automatically replicates each block three times to ensure fault tolerance and data reliability.

```

student207@hadoop-master:~$ hdfs dfs -stat %r /user/student207/file1.txt
3
student207@hadoop-master:~$ hdfs dfs -cat /user/student207/file1.txt
Hello Hadoop

```

Figure 10: Replication and content

```

student207@hadoop-master:~$ hdfs dfs -mkdir /user/student207/data
hdfs dfs -mkdir /user/student207/archive
student207@hadoop-master:~$ hdfs dfs -mv /user/student207/file1.txt /user/student207/data/
hdfs dfs -mv /user/student207/file2.txt /user/student207/archive/
student207@hadoop-master:~$ hdfs dfs -ls /user/student207/
Found 3 items
drwxr-xr-x - student207 hadoop_students      0 2026-02-24 14:21 /user/student207/archive
drwxr-xr-x - student207 hadoop_students      0 2026-02-24 14:21 /user/student207/data
-rw-r--r-- 3 student207 hadoop_students 10485760 2026-02-24 14:13 /user/student207/largefile.txt
student207@hadoop-master:~$

```

Figure 11: Files moved

13. View File Contents

Command Used:

```
hdfs dfs -cat /user/student207/file1.txt
```

Answer:

The file contents were displayed directly from HDFS without downloading the file locally which is "Hello Hadoop".

Explanation:

The `-cat` command reads file data stored in HDFS and prints it to the terminal.

14. Directory Structure and Move Files

Command Used:

```

hdfs dfs -mkdir /user/student207/data
hdfs dfs -mkdir /user/student207/archive
hdfs dfs -mv /user/student207/file1.txt /user/student207/data/
hdfs dfs -mv /user/student207/file2.txt /user/student207/archive/

```

Answer:

Directories were created and files were successfully moved into their respective folders.

Explanation:

The `-mkdir` command creates directories in HDFS, while `-mv` moves files within the distributed filesystem.

15. Change Replication Factor

Command Used:

```

hdfs dfs -setrep 2 /user/student207/largefile.txt
hdfs dfs -stat %r /user/student207/largefile.txt

```

```
student207@hadoop-master:~$ hdfs dfs -setrep 2 /user/student207/largefile.txt
Replication 2 set: /user/student207/largefile.txt
```

Figure 12: replication factor changed

Answer:

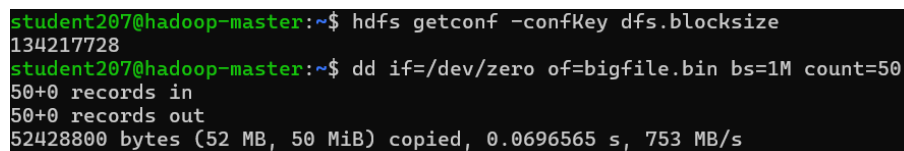
The replication factor of `largefile.txt` was successfully changed to 2.

Explanation:

The `-setrep` command modifies the number of replicas stored across DataNodes to control redundancy.

E. PHASE 3: REPLICATION & BLOCK ANALYSIS (25 MINUTES)

Challenge 3.1: Block Distribution



```
student207@hadoop-master:~$ hdfs getconf -confKey dfs.blocksize
134217728
student207@hadoop-master:~$ dd if=/dev/zero of=bigfile.bin bs=1M count=50
50+0 records in
50+0 records out
52428800 bytes (52 MB, 50 MiB) copied, 0.0696565 s, 753 MB/s
```

Figure 13: Block size configuration and 50MB file upload

16. Default Block Size

Command Used:

```
hdfs getconf -confKey dfs.blocksize
```

Answer:

The default HDFS block size is 134217728 bytes (128 MB).

Explanation:

HDFS stores files in blocks, and the default block size in this cluster is 128 MB, meaning files smaller than 128 MB occupy a single block.

17-18. 50MB File Block Analysis

Command Used:

```
dd if=/dev/zero of=bigfile.bin bs=1M count=50
hdfs dfs -put bigfile.bin /user/student207/
```

Answer:

The 50MB file was stored in a single HDFS block. Since the default block size is 128MB, the file size does not exceed one block.

Explanation:

HDFS splits files only when they exceed the configured block size. Because 50MB < 128MB, the file occupies exactly one block with 3 replicas (default replication factor).

Challenge 3.2: Replication Factor Experiments

19-23. Replication Analysis

```
student207@hadoop-master:~$ time hdfs dfs -setrep 1 /user/student207/bigfile.bin
Replication 1 set: /user/student207/bigfile.bin

real    0m1.707s
user    0m5.921s
sys     0m0.496s
student207@hadoop-master:~$ hdfs fsck /user/student207/bigfile.bin -files -blocks | grep -i "rep
Connecting to namenode via http://hadoop-master:9870/fsck?ugi=student207&files=1&blocks=1&path=%
/user/student207/bigfile.bin 52428800 bytes, replicated: replication=1, 1 block(s): OK
Replicated Blocks:
Minimally replicated blocks: 1 (100.0 %)
Over-replicated blocks: 0 (0.0 %)
Under-replicated blocks: 0 (0.0 %)
Mis-replicated blocks: 0 (0.0 %)
Default replication factor: 3
Average block replication: 1.0
Missing replicas: 0 (0.0 %)
Blocks queued for replication: 0
Blocks queued for replication: 0
student207@hadoop-master:~$ hdfs dfs -stat %r /user/student207/bigfile.bin
1
student207@hadoop-master:~$ time hdfs dfs -setrep 3 /user/student207/bigfile.bin
Replication 3 set: /user/student207/bigfile.bin

real    0m1.597s
user    0m5.229s
sys     0m0.439s
student207@hadoop-master:~$ hdfs dfs -stat %r /user/student207/bigfile.bin
3
student207@hadoop-master:~$ hdfs dfsadmin -report | grep -i "Under replicated blocks"
Under replicated blocks: 312
```

Figure 14: Replication factor modification and timing results

Command Used (Decrease Replication to 1):

```
time hdfs dfs -setrep 1 /user/student207/bigfile.bin
hdfs dfs -stat %r /user/student207/bigfile.bin
```

Answer:

The replication factor was successfully reduced to 1. The operation took approximately 1.707 seconds.

Explanation:

Decreasing replication removes extra block copies from DataNodes. This is generally fast since it only involves deleting replicas.

Command Used (Increase Replication to 3):

```
time hdfs dfs -setrep 3 /user/student207/bigfile.bin
hdfs dfs -stat %r /user/student207/bigfile.bin
```

Answer:

The replication factor was restored to 3. The operation took approximately 1.597 seconds.

Explanation:

Increasing replication requires copying blocks to additional DataNodes, which involves network transfer and synchronization.

Comparison:

The time difference between decreasing and increasing replication was minimal because the file contains only one block (50MB), making replication operations relatively fast.

Challenge 3.3: Rack Awareness

Command Used:

```
hdfs getconf -confKey net.topology.script.file.name
```

Answer:

Rack awareness is not configured in this cluster.

Explanation:

The property `net.topology.script.file.name` is missing, which means no rack topology script is defined. All DataNodes are treated as belonging to a single rack.

Block Placement Analysis:

```
hdfs fsck /user/student207/bigfile.bin -files -blocks -locations
```

The file has 3 replicas located on:

- 192.168.20.48
- 192.168.20.149
- 192.168.20.124

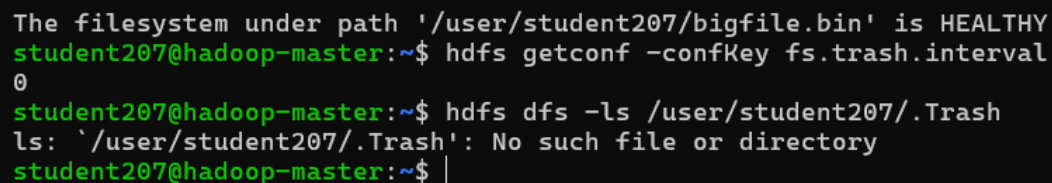
The replicas are placed on different DataNodes, ensuring fault tolerance. However, since the cluster has only one rack configured, the replicas are not distributed across multiple racks.

F. PHASE 4: TROUBLESHOOTING & ADVANCED COMMANDS

Scenario A: Deleted File Recovery

Command Used:

```
hdfs getconf -confKey fs.trash.interval
```



```
The filesystem under path '/user/student207/bigfile.bin' is HEALTHY
student207@hadoop-master:~$ hdfs getconf -confKey fs.trash.interval
0
student207@hadoop-master:~$ hdfs dfs -ls /user/student207/.Trash
ls: '/user/student207/.Trash': No such file or directory
student207@hadoop-master:~$ |
```

Figure 15: Verification that HDFS Trash is disabled

Answer:

Trash is disabled on this cluster.

Explanation:

The configuration value returned is 0, which means deleted files are permanently removed immediately and cannot be recovered from a Trash directory.

Scenario B: Small Files Problem

Answer:

100 small files consume significantly more NameNode memory than one large file of equivalent total size.

Explanation:

Each file requires separate metadata (inode and block information) stored in NameNode memory. Approximately 300–500 bytes are used per file, meaning 100 small files consume around 30–50 KB, while a single large file uses only a few hundred bytes.

Scenario C: Corrupted Block Simulation

Command Used:

```
hdfs debug recoverLease /path/to/file
```

Answer:

The recoverLease command forces recovery of a file that was left open due to client failure.

Explanation:

If a client crashes while writing, the file lease remains active. The recoverLease command allows the NameNode to close the file properly and make it accessible again.

Advanced Discovery Questions (27-30)

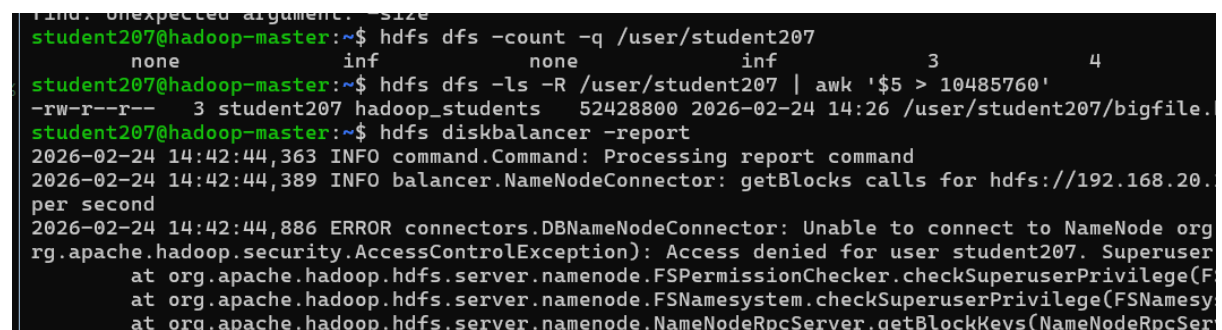
Answer:

Additional administrative commands such as quota checks, large file searches, disk balancer status, and NameNode web interface analysis can be performed using HDFS administration tools.

Examples:

```
hdfs dfs -count -q /user/student207
hdfs dfs -find /user/student207 -size +10M
hdfs diskbalancer -report
```

Challenge 4.2: Advanced Discovery



```
find: unexpected argument: -size
student207@hadoop-master:~$ hdfs dfs -count -q /user/student207
      none      inf      none      inf      3      4
student207@hadoop-master:~$ hdfs dfs -ls -R /user/student207 | awk '$5 > 10485760'
-rw-r--r--   3 student207  hadoop_students   52428800  2026-02-24 14:26 /user/student207/bigfile.
student207@hadoop-master:~$ hdfs diskbalancer -report
2026-02-24 14:42:44,363 INFO command.Command: Processing report command
2026-02-24 14:42:44,389 INFO balancer.NameNodeConnector: getBlocks calls for hdfs://192.168.20.
per second
2026-02-24 14:42:44,886 ERROR connectors.DBNameNodeConnector: Unable to connect to NameNode org
rg.apache.hadoop.security.AccessControlException): Access denied for user student207. Superuser
at org.apache.hadoop.hdfs.server.namenode.FSPermissionChecker.checkSuperuserPrivilege(F
at org.apache.hadoop.hdfs.server.namenode.FSNamesystem.checkSuperuserPrivilege(FSNamesy
at org.apache.hadoop.hdfs.server.namenode.NameNodeRpcServer.getBlockKeys(NameNodeRpcSer
```

27. HDFS Quotas on Home Directory

Command Used:

```
hdfs dfs -count -q /user/student207
```

Answer:

No namespace or space quotas are configured for the home directory.

Explanation:

The command returned **none** for quota limits, indicating that there are no restrictions on the number of files or storage space usage.

28. Files Larger Than 10MB

Command Used:

```
hdfs dfs -ls -R /user/student207 | awk '$5 > 10485760'
```

Answer:

The file `bigfile.bin` (50MB) was identified as larger than 10MB.

Explanation:

The recursive listing combined with size filtering allowed identification of files exceeding the 10MB threshold.

29. Disk Balancer Status (Hadoop 3.x)

Command Used:

```
hdfs diskbalancer -report
```

Answer:

Disk balancer operations require superuser privileges and cannot be executed by a regular user.

Explanation:

The command returned an access denied error, indicating that disk balancing is an administrative operation restricted to HDFS superusers.

30. NameNode Web UI Analysis

Accessed URL:

```
http://192.168.20.153:9870
```

Answer:

The NameNode web interface provides real-time cluster monitoring and filesystem browsing.

Explanation:

The dashboard displays live DataNodes, storage capacity statistics, block information, and allows browsing of the HDFS filesystem structure.

G. LAB REPORT REQUIREMENTS

1. Cluster Topology Diagram (Nodes and Roles)

NameNode (Master Node):

- **Hostname:** hadoop-master
- **IP Address:** 192.168.20.153
- **Port:** 9000
- **Role:** Manages HDFS metadata and namespace

DataNodes (Worker Nodes):

- DataNode – 192.168.20.115:9866
- DataNode – 192.168.20.48:9866
- DataNode – 192.168.20.124:9866
- Additional DataNode (Total DataNodes reported: 4)

Cluster Summary:

- HDFS Version: 3.3.6
- Default Replication Factor: 3
- Number of Racks: 1 (Rack awareness not configured)

2. Replication Analysis (Block Locations for 50MB File)

Command Used:

```
hdfs fsck /user/student207/bigfile.bin -files -blocks -locations
```

Analysis:

The 50MB file is stored in one HDFS block (since 50MB < 128MB default block size). The block has a replication factor of 3 with live replicas located on:

```

-3d2483ae-9232-48af-8a70-5e2de15a3308,DISK]]

Status: HEALTHY
Number of data-nodes: 4
Number of racks: 1
Total dirs: 0
Total symlinks: 0

Replicated Blocks:
Total size: 52428800 B
Total files: 1
Total blocks (validated): 1 (avg. block size 52428800 B)
Minimally replicated blocks: 1 (100.0 %)
Over-replicated blocks: 0 (0.0 %)
Under-replicated blocks: 0 (0.0 %)
Mis-replicated blocks: 0 (0.0 %)
Default replication factor: 3
Average block replication: 3.0
Missing blocks: 0
Corrupt blocks: 0
Missing replicas: 0 (0.0 %)
Blocks queued for replication: 0

Erasure Coded Block Groups:
Total size: 0 B
Total files: 0
Total block groups (validated): 0
Minimally erasure-coded block groups: 0
Over-erasure-coded block groups: 0
Under-erasure-coded block groups: 0
Unsatisfactory placement block groups: 0
Average block group size: 0.0
Missing block groups: 0
Corrupt block groups: 0
Missing internal blocks: 0
Blocks queued for replication: 0

FSCK ended at Tue Feb 24 14:57:42 UTC 2026 in 0 milliseconds

```

Figure 16: Enter Caption

- 192.168.20.115:9866
- 192.168.20.48:9866
- 192.168.20.124:9866

The filesystem status is **HEALTHY** with no missing or corrupt blocks, confirming correct HDFS replication behavior.

3. Timing Comparison (Replication Increase vs Decrease)

Commands Used:

```
time hdfs dfs -setrep 1 /user/student207/bigfile.bin
time hdfs dfs -setrep 3 /user/student207/bigfile.bin
```

Table 1: Replication timing comparison for `bigfile.bin`

Operation	Time (s)	Observation / Reason
Decrease replication (3 → 1)	1.707	Fast operation because extra replicas are removed from DataNodes.
Increase replication (1 → 3)	1.597	Requires copying blocks to additional DataNodes over the network.

Conclusion: The timing difference is minimal because the file occupies only one block (50MB), making replication changes fast in both cases.

4. Troubleshooting Log (Scenarios A, B, C)

Scenario A: “I deleted an important file!”

Command Used:

```
hdfs getconf -confKey fs.trash.interval
```

Result: 0 (Trash disabled)

Conclusion: Deleted files are permanently removed immediately and cannot be recovered.

Scenario B: “Small files problem”

Explanation:

Each file stored in HDFS consumes NameNode memory for metadata (inode and block information). Approximately 300–500 bytes are used per file.

Therefore:

- 100 small files $\approx$ 30–50KB metadata usage
- 1 large file uses only a few hundred bytes

Small files significantly increase NameNode memory consumption.

Scenario C: “Corrupted block simulation”

Command Used:

```
hdfs debug recoverLease /path/to/file
```

Explanation:

The recoverLease command forces lease recovery when a client crashes during file writing. It allows the NameNode to close the file properly and restore accessibility.

5. Command Reference: Top 10 Useful HDFS Commands

1. List files

```
hdfs dfs -ls /
```

2. Create directory

```
hdfs dfs -mkdir /user/student207/data
```

3. Upload file

```
hdfs dfs -put file.txt /user/student207/
```

4. Download file

```
hdfs dfs -get /user/student207/file.txt .
```

5. View content

```
hdfs dfs -cat /user/student207/file.txt
```

6. Move file

```
hdfs dfs -mv file.txt archive/
```

7. Delete file

```
hdfs dfs -rm file.txt
```

8. Check replication

```
hdfs dfs -stat %r bigfile.bin
```

9. Change replication

```
hdfs dfs -setrep 2 bigfile.bin
```

10. Cluster health check

```
hdfs fsck /user/student207 -files -blocks -locations
```